

SENA

TECNÓLOGO ANÁLISIS Y DESARROLLO DE SOFTWARE

MATERIA:

Actividad de proyecto 4

Evidencia:

Evidencia GA4-220501095-AA3-EV03.docx Diagrama de despliegue para caso de estudio y proyecto de software.

Estudiante:

Gerson Jonnathan López Oviedo – 3186647

DOCENTE:

Arnaldo Alfonso Montiel Brun

BOGOTÁ D.C.

7 de diciembre 2025

INDICE

1. INTRODUCCIÓN
2. LINK DIAGRAMA DE PROCESOS APP TRUEQUE MATCH
3. Link diagrama de despliegue app TRUEQUE MATCH
4. DESCRIPCIÓN GENERAL DEL DIAGRAMA
5. COMPONENTES DEL SISTEMA
 - 5.1. Cliente (Móvil/Web)
 - 5.2. Repositorio de Código (Githuub)
 - 5.3. Servidor de Aplicaciones
 - 5.4. Servidor de BD
 - 5.5. Servidor Web
 - 5.6. Servidor de Archivos
 - 5.7. Servicios Externos
6. FLUJO DE COMUNICACIÓN
7. PROTOCOLOS DE COMUNICACIÓN
8. REQUISITOS NO FUNCIONALES CUBIERTOS
9. JUSTIFICACIÓN DE DECISIONES TÉCNICAS
10. CONCLUSIONES
11. REFERENCIAS

1. INTRODUCCIÓN

El diagrama de despliegue es una herramienta fundamental en el desarrollo de software que permite visualizar la arquitectura física de un sistema, mostrando cómo los diferentes componentes de software se distribuyen en los nodos de hardware y cómo estos se comunican entre sí. En el contexto del desarrollo de aplicaciones modernas, comprender esta distribución es esencial para garantizar el correcto funcionamiento, escalabilidad y mantenimiento del sistema.

En el presente documento se presenta el diagrama de despliegue del proyecto TRUEQUE MATCH, una aplicación de economía colaborativa diseñada para facilitar el intercambio de productos, servicios, conocimientos y experiencias sin el uso de dinero. Este diagrama representa la arquitectura técnica que soporta todas las funcionalidades definidas en las fases previas de análisis y diseño del proyecto.

El diagrama de despliegue de TRUEQUE MATCH ilustra la disposición de los artefactos de software en los diferentes nodos físicos y lógicos del sistema, incluyendo:

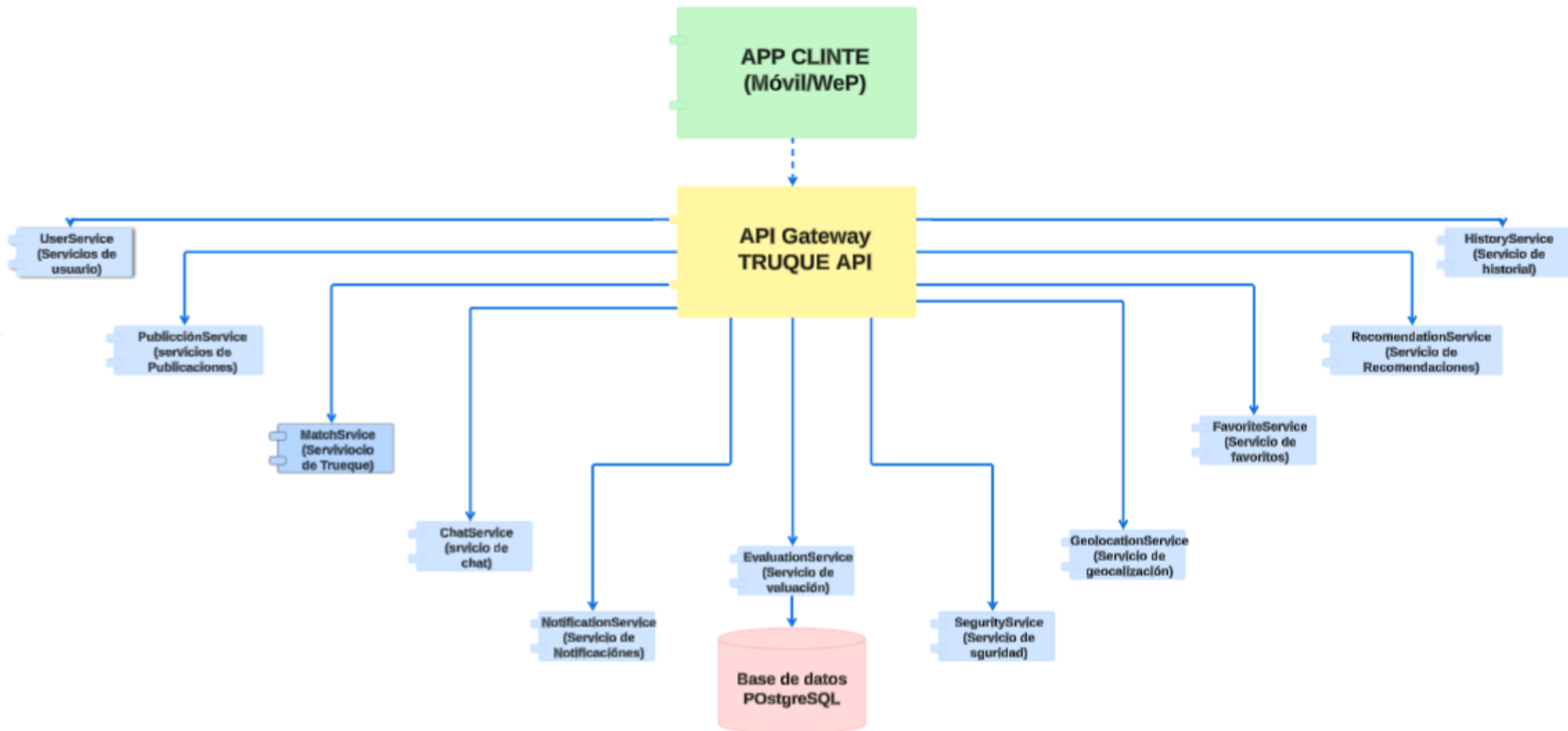
- Cliente móvil y web: interfaces de usuario que permiten la interacción con la plataforma
- Servidor de aplicaciones: núcleo de la lógica de negocio que procesa las peticiones
- Servidor de base de datos: almacenamiento persistente de toda la información del sistema
- Repositorio de código: control de versiones y colaboración en el desarrollo
- Servicios externos: componentes de terceros que complementan la funcionalidad

Este diagrama no solo cumple con los requisitos técnicos establecidos, sino que también responde a los requisitos funcionales y no funcionales definidos previamente, asegurando que la arquitectura propuesta sea viable, escalable y adaptada a las necesidades reales de los usuarios de TRUEQUE MATCH.

2. LINK DIAGRAMA DE PROCESOS APP TRUEQUE MATCH:

https://lucid.app/lucidchart/434fe6ea-7396-45ed-af5c-95e84f6975b5/edit?viewport_loc=187%2C-3475%2C5425%2C2396%2C0_0&invitationId=inv_cfa566e4-62b1-4eaf-8994-2692f940fc18

DIAGRAMA DE PROCESOS APP TRUEQUE MATCH



3. LINK DIAGRAMA DE DESPLIEGUE APP TRUEQUE MATCH:

https://lucid.app/lucidchart/2a3e9261-c7d4-4ebc-8bbd-1ca6853dee69/edit?viewport_loc=-2025%2C-1682%2C6762%2C2987%2C0_0&invitationId=inv_7f117844-ef2e-411d-ab59-50ca73a812a4

Diagrama de despliegue TRUEQUE MATCH

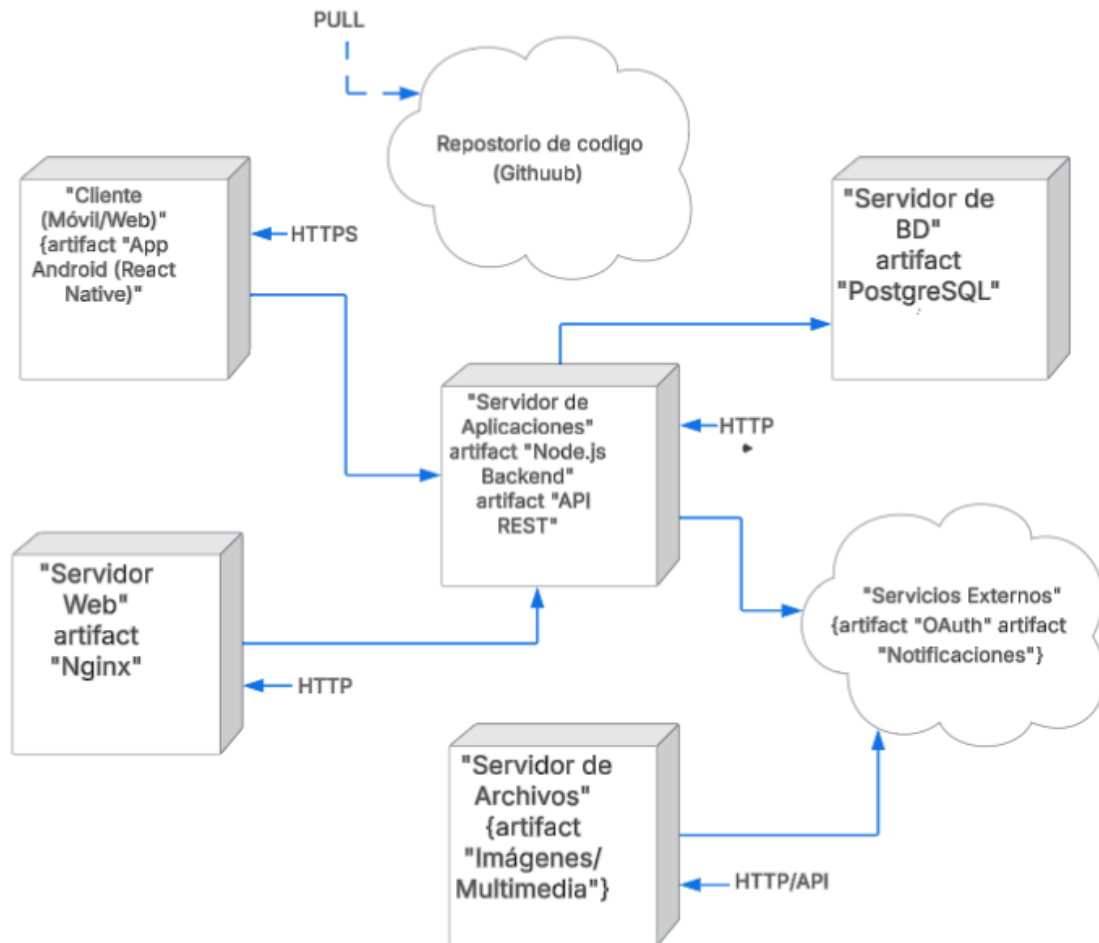


DIAGRAMA DE DESPLIEGUE - TRUEQUE MATCH

4. DESCRIPCIÓN GENERAL DEL DIAGRAMA

El diagrama de despliegue de TRUEQUE MATCH representa una arquitectura distribuida basada en el modelo cliente-servidor de tres capas (presentación, lógica de negocio y datos), con integración de servicios externos y repositorio de código centralizado

5. COMPONENTES DEL SISTEMA

5.1. Cliente (Móvil/Web)

Artifact: "App Android (React Native)"

Descripción:

Representa la interfaz de usuario con la que los usuarios finales interactúan directamente. Esta capa de presentación está disponible en dos modalidades:

- **Aplicación móvil:** Desarrollada en React Native para Android, permite acceso desde dispositivos móviles
- **Aplicación web:** Desarrollada en React.js, accesible desde navegadores web

Funcionalidades que ejecuta:

- Registro e inicio de sesión de usuarios
- Visualización y creación de publicaciones de trueque
- Envío y recepción de propuestas
- Chat y negociación entre usuarios
- Gestión de perfil y configuraciones
- Visualización de historial y evaluaciones

Comunicación:

Se conecta mediante **HTTPS** (protocolo seguro) con el servidor de aplicaciones para realizar peticiones y recibir respuestas.

Tecnologías:

- React Native (móvil)

- React.js (web)
- HTTPS para comunicación segura

5.2. Repositorio de Código (Github)

Artifact: Control de versiones

Descripción:

Nodo en la nube que almacena todo el código fuente del proyecto, permitiendo:

- Control de versiones del código
- Colaboración entre desarrolladores
- Historial de cambios y modificaciones
- Gestión de ramas de desarrollo

Comunicación:

Conectado mediante **PULL** con el cliente para obtener actualizaciones y mediante **HTTP** con el servidor de aplicaciones para despliegue continuo.

Importancia:

- Garantiza la trazabilidad del código
- Facilita el trabajo colaborativo
- Permite reversión de cambios en caso de errores
- Documenta la evolución del proyecto

5.3. Servidor de Aplicaciones

Artifact: "Node.js Backend"

Artifact: "API REST"

Descripción:

Es el núcleo del sistema, donde reside toda la lógica de negocio. Este servidor procesa

las peticiones del cliente, ejecuta las operaciones necesarias y coordina la comunicación con la base de datos y servicios externos.

Funciones principales:

- Autenticación y autorización de usuarios
- Procesamiento de peticiones HTTP
- Validación de datos
- Implementación de reglas de negocio
- Gestión de sesiones
- Coordinación con base de datos
- Integración con servicios externos

Tecnologías:

- **Node.js:** Entorno de ejecución JavaScript
- **Express.js:** Framework para API REST
- **JWT:** Para autenticación segura
- **Comunicación:**
- **HTTP/HTTPS** con el cliente
- **HTTP** con el servidor de base de datos
- **HTTP/API** con servicios externos
- **HTTP** con servidor web (Nginx)
- **HTTP** con servidor de archivos

5.4. Servidor de BD

Artifact: "PostgreSQL"

Descripción:

Sistema de gestión de base de datos relacional que almacena de forma persistente toda la información del sistema.

Datos almacenados:

- **Usuarios:** información personal, credenciales, preferencias
- **Ofertas:** publicaciones de productos, servicios, conocimientos
- **Trueques:** registros de intercambios realizados
- **Propuestas:** solicitudes de intercambio entre usuarios
- **Evaluaciones:** calificaciones y comentarios
- **Notificaciones:** alertas y mensajes del sistema
- **Reportes:** denuncias y registros de seguridad

Comunicación:

Conectado mediante **HTTP** con el servidor de aplicaciones. Recibe consultas SQL y retorna resultados.

Características:

- PostgreSQL (base de datos relacional robusta)
- Normalizado según formas normales 1FN, 2FN y 3FN
- Respaldo automático de datos
- Integridad referencial garantizada

5.5. Servidor Web

Artifact: "Nginx"

Descripción:

Servidor web que actúa como proxy inverso, balanceador de carga y servidor de archivos estáticos.

Funciones:

- Servir archivos estáticos (HTML, CSS, JS)
- Redirección de tráfico al servidor de aplicaciones
- Balanceo de carga en caso de múltiples instancias
- Implementación de caché para mejorar rendimiento
- Compresión de archivos para optimizar transferencia

Comunicación:

Conectado mediante **HTTP** con el servidor de aplicaciones.

5.6. Servidor de Archivos

Artifact: "Imágenes/Multimedia"

Descripción:

Nodo dedicado al almacenamiento y gestión de archivos multimedia subidos por los usuarios.

Contenido almacenado:

- Fotos de perfil de usuarios
- Imágenes de productos y servicios ofertados
- Documentos adjuntos en publicaciones
- Archivos multimedia relacionados con trueques

Comunicación:

Conectado mediante **HTTP/API** con el servidor de aplicaciones y con servicios externos para procesamiento de imágenes.

Características:

- Almacenamiento optimizado
- Compresión automática de imágenes
- Control de tamaño y formato de archivos
- Respaldo periódico

5.7. Servicios Externos

Artifact: "OAuth"**Artifact: "Notificaciones"****Descripción:**

Conjunto de servicios de terceros que complementan la funcionalidad del sistema.

Servicios incluidos:**OAuth (Autenticación externa):**

- Permite inicio de sesión con cuentas de Google, Facebook, etc.
- Simplifica el proceso de registro
- Aumenta la seguridad mediante proveedores confiables

Sistema de Notificaciones:

- Envío de notificaciones push a dispositivos móviles
- Correos electrónicos de confirmación y alertas
- SMS para verificación de número telefónico (opcional)

Geolocalización:

- API de mapas para mostrar ubicación de ofertas
- Cálculo de distancias entre usuarios

- Filtrado por proximidad geográfica

Comunicación:

Conectado mediante **HTTP/API** con el servidor de aplicaciones y con el servidor de archivos.

6. FLUJO DE COMUNICACIÓN

Escenario 1: Usuario registra una cuenta

1. El **Cliente** envía datos de registro vía **HTTPS** al **Servidor de Aplicaciones**
2. El **Servidor de Aplicaciones** valida la información
3. Se conecta al **Servidor de BD** para verificar que el correo no esté registrado
4. Si es válido, se guarda el nuevo usuario en **PostgreSQL**
5. Se envía correo de verificación mediante **Servicios Externos (Notificaciones)**
6. Se retorna confirmación al **Cliente**

Escenario 2: Usuario publica una oferta de trueque

1. El **Cliente** envía datos de la publicación (título, descripción, imágenes) al **Servidor de Aplicaciones**
2. Las imágenes se envían al **Servidor de Archivos** para almacenamiento
3. El **Servidor de Aplicaciones** procesa la información y la guarda en **PostgreSQL**
4. Se genera notificación a usuarios interesados mediante **Servicios Externos**
5. La publicación queda visible en el feed del **Cliente**

Escenario 3: Usuario recibe una propuesta de trueque

1. El **Servidor de Aplicaciones** detecta una nueva propuesta en **PostgreSQL**
2. Se genera notificación push mediante **Servicios Externos**
3. El **Cliente** recibe la notificación en tiempo real
4. El usuario accede al detalle de la propuesta desde el **Cliente**
5. El **Servidor de Aplicaciones** recupera los datos completos desde **PostgreSQL**

7. PROTOCOLOS DE COMUNICACIÓN

HTTPS (HTTP Secure)

- Utilizado entre Cliente y Servidor de Aplicaciones
- Garantiza cifrado de datos sensibles (contraseñas, información personal)
- Certificado SSL para autenticación del servidor

HTTP

- Utilizado para comunicaciones internas entre servidores
- Optimizado para velocidad en red local o privada

PULL

- Mecanismo de actualización desde el repositorio de código
- Permite obtener la última versión del código fuente

API REST

- Interfaz estándar de comunicación basada en HTTP
- Métodos: GET, POST, PUT, DELETE
- Formato de datos: JSON

8. REQUISITOS NO FUNCIONALES CUBIERTOS

El diagrama de despliegue garantiza el cumplimiento de los siguientes requisitos no funcionales:

RNF01 - Rendimiento

- Servidor de aplicaciones optimizado con Node.js
- Servidor web Nginx para caché y balanceo de carga
- Servidor de archivos separado para no sobrecargar el servidor principal

RNF02 - Disponibilidad

- Arquitectura distribuida permite alta disponibilidad
- Posibilidad de replicación del servidor de aplicaciones
- Respaldo automático de base de datos

RNF03 - Seguridad

- Comunicación HTTPS entre cliente y servidor
- Autenticación mediante JWT
- OAuth para integración segura con servicios externos
- PostgreSQL con políticas de integridad y seguridad

RNF04 - Escalabilidad

- Arquitectura modular permite agregar nodos según demanda
- Servidor web puede actuar como balanceador de carga
- Base de datos PostgreSQL soporta gran volumen de datos

RNF05 - Mantenibilidad

- Código centralizado en GitHub
- Separación clara de responsabilidades (capas)
- Documentación técnica completa

RNF06 - Portabilidad

- Cliente disponible en múltiples plataformas (móvil y web)
- Uso de tecnologías multiplataforma (React Native, Node.js)
- Despliegue posible en diversos proveedores de nube

9. JUSTIFICACIÓN DE DECISIONES TÉCNICAS

¿Por qué Node.js?

- Lenguaje JavaScript unificado para frontend y backend

- Alto rendimiento en operaciones asíncronas
- Amplia comunidad y ecosistema de librerías
- Ideal para aplicaciones en tiempo real (chat, notificaciones)

¿Por qué PostgreSQL?

- Base de datos relacional robusta y confiable
- Software libre (sin costo de licencias)
- Soporta gran volumen de datos y transacciones concurrentes
- Cumplimiento de estándares SQL

¿Por qué React Native?

- Permite desarrollo multiplataforma con un solo código base
- Comunidad activa y bien documentada
- Rendimiento cercano a apps nativas
- Reducción de tiempo y costo de desarrollo

¿Por qué Nginx?

- Servidor web ligero y de alto rendimiento
- Software libre
- Excelente para servir archivos estáticos
- Funciona eficientemente como proxy inverso

¿Por qué GitHub?

- Plataforma estándar de la industria
- Gratuito para proyectos de código abierto
- Integración con herramientas de despliegue continuo
- Facilita colaboración y revisión de código

10. CONCLUSIONES

El diagrama de despliegue presentado para el proyecto TRUEQUE MATCH refleja una arquitectura distribuida, escalable y alineada con las mejores prácticas de desarrollo de software moderno. La separación de responsabilidades en diferentes nodos garantiza que el sistema sea mantenible, eficiente y preparado para crecer conforme aumente la demanda de usuarios.

La elección de tecnologías de código abierto como Node.js, PostgreSQL, React Native y Nginx demuestra que es posible construir soluciones robustas y profesionales sin depender de licencias costosas, lo cual es fundamental en un contexto académico y de desarrollo con recursos limitados. Esta decisión técnica no solo reduce costos, sino que también facilita el aprendizaje y la experimentación sin barreras económicas.

El diagrama cumple con todos los requisitos funcionales y no funcionales establecidos en las fases previas del proyecto:

- **Funcionalidad completa:** Todos los casos de uso definidos pueden ejecutarse correctamente en esta arquitectura
- **Rendimiento óptimo:** La separación de servidores y el uso de tecnologías eficientes garantizan tiempos de respuesta inferiores a 3 segundos
- **Alta disponibilidad:** La arquitectura permite replicación y balanceo de carga para mantener el sistema operativo el 99% del tiempo
- **Seguridad robusta:** El uso de HTTPS, autenticación JWT y OAuth protege la información sensible de los usuarios
- **Escalabilidad efectiva:** El sistema puede crecer agregando más nodos según sea necesario sin afectar el funcionamiento actual

Además, la integración con servicios externos como sistemas de notificaciones y autenticación OAuth enriquece la experiencia del usuario sin complicar excesivamente la arquitectura interna. El repositorio de código centralizado en GitHub garantiza la trazabilidad del desarrollo y facilita la colaboración, aspectos esenciales en proyectos académicos y profesionales.

Este diagrama de despliegue no es solo un requisito técnico cumplido, sino una herramienta de planificación que guiará las siguientes fases de implementación del

proyecto TRUEQUE MATCH. Representa la base arquitectónica sobre la cual se construirá una aplicación que tiene el potencial de generar un impacto social real, promoviendo la economía colaborativa y rescatando valores ancestrales de intercambio comunitario mediante la tecnología moderna.

Finalmente, este trabajo demuestra que con una planificación adecuada, conocimiento técnico y el uso estratégico de herramientas gratuitas, es posible diseñar sistemas de software de calidad profesional que cumplan con estándares internacionales y que estén preparados para enfrentar los desafíos del mundo digital actual.

11. REFERENCIAS

- Node.js Foundation. (s.f.). *Node.js documentation*. Recuperado de <https://nodejs.org>
- PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. Recuperado de <https://www.postgresql.org>
- Meta. (s.f.). *React Native documentation*. Recuperado de <https://reactnative.dev>
- Nginx Software, Inc. (s.f.). *Nginx documentation*. Recuperado de <https://nginx.org/en/docs/>
- Object Management Group (OMG). (2017). *UML deployment diagram specification*. Recuperado de <https://www.omg.org/spec/UML>
- IEEE. (1998). *IEEE Std 830-1998: Recommended practice for software requirements specifications*. IEEE.
- López Oviedo, G. J. (2025). *Documentación del proyecto TRUEQUE MATCH (evidencias académicas)*. Material inédito.

Enlaces diagramas:

Enlace al diagrama de despliegue (Lucidchart):

<https://lucid.app/lucidchart/2a3e9261-c7d4-4ebc-8bbd-1ca6853dee69/edit>

Enlace al diagrama de procesos:

<https://lucid.app/lucidchart/434fe6ea-7396-45ed-af5c-95e84f6975b5/edit>

